

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 1
MODUL 14
“ SORTING ”



DISUSUN OLEH:
Harding Rafif Dzakwan Permana
10908230018
S1 IF-12-02
DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025/2026

DASAR TEORI

Dasar teori sorting adalah proses mengurutkan sekumpulan data ke dalam urutan tertentu, baik secara menaik (ascending) maupun menurun (descending). Pengurutan data dilakukan agar proses pencarian dan pengolahan data menjadi lebih mudah dan efisien. Salah satu metode sorting adalah Selection Sort, yaitu algoritma yang bekerja dengan mencari nilai terkecil atau terbesar dari kumpulan data, lalu menukarnya ke posisi yang sesuai secara berulang hingga seluruh data terurut. Contohnya, data **7 3 9 1** setelah diurutkan secara ascending menjadi **1 3 7 9**.

Metode lainnya adalah Insertion Sort, yaitu algoritma pengurutan dengan cara menyisipkan data pada posisi yang tepat di bagian data yang sudah terurut. Algoritma ini membandingkan elemen yang sedang diproses dengan elemen sebelumnya, kemudian menggeser data hingga ditemukan posisi yang sesuai. Contohnya, data **31 13 25 43 1** setelah diurutkan menggunakan insertion sort menjadi **1 13 25 31 43**. Metode ini cukup sederhana dan efektif digunakan pada jumlah data yang tidak terlalu besar.

Latihan Soal

Selection Sort

1. Rumah Kerabat

Source Code:

```
package main
import "fmt"
const NMAX int = 1000
type tabInt [NMAX]int
func selectionSort(T *tabInt, n int) {
    var i, j, idxMin, temp int

    for i = 0; i < n-1; i++ {
        idxMin = i

        for j = i + 1; j < n; j++ {
            if T[j] < T[idxMin] {
                idxMin = j
            }
        }
        temp = T[i]
        T[i] = T[idxMin]
        T[idxMin] = temp
    }
}
func main() {
    var n, m int
    var T tabInt
    fmt.Print("Masukan angka :")
    fmt.Scan(&n)

    for i := 0; i < n; i++ {

        fmt.Scan(&m)

        for j := 0; j < m; j++ {
            fmt.Scan(&T[j])
        }

        selectionSort(&T, m)

        for j := 0; j < m; j++ {
            fmt.Print(T[j])

            if j < m-1 {
                fmt.Print(" ")
            }
        }
    }
}
```

```

        fmt.Println()
    }
}

```

Output:

```

modul 14 > nomori.go > selectionSort
1 package main
2
3 import "fmt"
4
5 const NMAX int = 1000
6
7 type tabInt [NMAX]int
8 func selectionSort(T *tabInt, n int) {
9     var i, j, idxMin, temp int
10
11     for i = 0; i < n-1; i++ {
12         idxMin = i
13
14         for j = i + 1; j < n; j++ {
15             if T[j] < T[idxMin] {
16                 idxMin = j
17             }
18         }
19
20         temp = T[i]
21         T[i] = T[idxMin]
22         T[idxMin] = temp
23     }
24 }
25
26 func main() {
27     var n, m int

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

Masukan angka :3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9
PS D:\vscode test\Alpro Semester 2>

```

Deskripsi Program:

Program rumahkerabat yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma selection sort.

Masukan dimulai dengan sebuah integer n ($0 < n < 1000$), banyaknya daerah kerabat Hercules tinggal. Isi n baris berikutnya selalu dimulai dengan sebuah integer m ($0 < m < 1000000$) yang menyatakan banyaknya rumah kerabat di daerah tersebut, diikuti dengan rangkaian bilangan bulat positif, nomor rumah para kerabat.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar di masingmasing daerah

const NMAX int = 1000 kapasitas maksimum array

type tabInt [NMAX]int tipe array integer

Procedure untuk mengurutkan data dengan selection sort

func selectionSort(T *tabInt, n int) {

var i, j, idxMin, temp int variabel bantu

for i = 0; i < n-1; i++ { perulangan untuk menentukan posisi data

idxMin = i anggap elemen pertama adalah yang terkecil

for j = i + 1; j < n; j++ { mencari data paling kecil

if T[j] < T[idxMin] { jika ditemukan data

```

lebih kecil
                                idxMin = j    simpan indeks data
terkecil
                                }
                                }

                                proses pertukaran data
                                temp = T[i]
                                T[i] = T[idxMin]
                                T[idxMin] = temp
                                }
                                }
func main() {
    var n, m int    n = jumlah daerah, m = jumlah rumah
    var T tabInt    array untuk menyimpan nomor rumah

    fmt.Scan(&n)    input jumlah daerah

    for i := 0; i < n; i++ {    perulangan untuk setiap
daerah

        fmt.Scan(&m)    input jumlah rumah kerabat

        for j := 0; j < m; j++ {    input nomor rumah
            fmt.Scan(&T[j])
        }

        selectionSort(&T, m)    memanggil procedure
sorting

        for j := 0; j < m; j++ {    menampilkan hasil
sorting
            fmt.Print(T[j])

            if j < m-1 {    agar tidak ada spasi di
akhir
                fmt.Print(" ")
            }
        }

        fmt.Println()    pindah baris untuk daerah
berikutnya
    }

```

2. Kerabat Dekat

Source Code:

```
package main
import "fmt"
const NMAX int = 1000
type tabInt [NMAX]int
func selectionSortAsc(T *tabInt, n int) {
    var i, j, idxMin, temp int

    for i = 0; i < n-1; i++ {
        idxMin = i

        for j = i + 1; j < n; j++ {
            if T[j] < T[idxMin] {
                idxMin = j
            }
        }
        temp = T[i]
        T[i] = T[idxMin]
        T[idxMin] = temp
    }
}
func selectionSortDesc(T *tabInt, n int) {
    var i, j, idxMax, temp int

    for i = 0; i < n-1; i++ {
        idxMax = i

        for j = i + 1; j < n; j++ {
            if T[j] > T[idxMax] {
                idxMax = j
            }
        }
        temp = T[i]
        T[i] = T[idxMax]
        T[idxMax] = temp
    }
}
func main() {
    var n, m, x int
    var ganjil, genap tabInt
    var nganjil, ngenap int
    fmt.Scan(&n)
    for i := 0; i < n; i++ {

        nganjil = 0
        ngenap = 0

        fmt.Scan(&m)
```

```

        for j := 0; j < m; j++ {
            fmt.Scan(&x)

            if x%2 == 1 {
                ganjil[nganjil] = x
                nganjil++
            } else {
                genap[ngenap] = x
                ngenap++
            }
        }
        selectionSortAsc(&ganjil, nganjil)
        selectionSortDesc(&genap, ngenap)

        for j := 0; j < nganjil; j++ {
            fmt.Print(ganjil[j], " ")
        }

        for j := 0; j < ngenap; j++ {
            fmt.Print(genap[j])

            if j < ngenap-1 {
                fmt.Print(" ")
            }
        }

        fmt.Println()
    }
}

```

Output:

```

modul14 > nomor2.go > ...
1  package main
2
3  import "fmt"
4
5  const NMAX int = 1000
6
7  type tabInt [NMAX]int
8
9  func selectionSortAsc(T *tabInt, n int) {
10     var i, j, idxMin, temp int
11
12     for i = 0; i < n-1; i++ {
13         idxMin = i
14
15         for j = i + 1; j < n; j++ {
16             if T[j] < T[idxMin] {
17                 idxMin = j
18             }
19         }
20
21         temp = T[i]
22         T[i] = T[idxMin]
23         T[idxMin] = temp
24     }
25 }

```

PROBLEMS 10 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS D:\vscode test\Alpro Semester 2> go run "d:\vscode test\Alpro Semester 2\modul 14\nomor2.go"
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4
PS D:\vscode test\Alpro Semester 2>

```

Ln 5, Col 23 Tab Size: 4 UTF-8 CRLF 1.25.1 69 Go Live 69 Go Live Prettier

Deskripsi Program:

Program kerabat dekat yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil

Program dimulai dengan **const NMAX int = 1000** yang digunakan sebagai kapasitas maksimum array, lalu **type tabInt [NMAX]int** digunakan untuk membuat tipe data array integer. Function **selectionSortAsc** berfungsi mengurutkan data secara ascending (kecil ke besar) dengan metode selection sort, yaitu mencari nilai terkecil lalu menukarnya ke posisi depan. Variabel **idxMin** digunakan untuk menyimpan indeks nilai terkecil, sedangkan **temp** dipakai untuk pertukaran data. Function **selectionSortDesc** bekerja hampir sama, tetapi digunakan untuk mengurutkan data secara descending (besar ke kecil) dengan bantuan variabel **idxMax** untuk mencari nilai terbesar.

Pada fungsi utama **main()**, program membaca jumlah test case menggunakan **fmt.Scan(&n)**. Kemudian program melakukan perulangan sebanyak jumlah test case. Di setiap perulangan, program membaca jumlah data **m**, lalu membaca angka satu per satu menggunakan **fmt.Scan(&x)**. Jika angka memenuhi kondisi **x%2 == 1**, maka angka dianggap ganjil dan disimpan ke array **ganjil**, sedangkan selain itu disimpan ke array **genap**. Setelah semua data dipisahkan, array ganjil diurutkan menggunakan **selectionSortAsc(&ganjil, nganjil)**, sedangkan array genap diurutkan menggunakan **selectionSortDesc(&genap, ngenap)**. Terakhir program mencetak seluruh bilangan ganjil terlebih dahulu lalu dilanjutkan dengan bilangan genap yang sudah terurut.

3. Median

Source Code:

```
package main
import "fmt"
const NMAX = 1000000
type tabInt [NMAX]int
func insertionSort(T *tabInt, n int) {
    var i, pass, temp int

    for pass = 1; pass < n; pass++ {
        temp = T[pass]
        i = pass

        for i > 0 && temp < T[i-1] {
            T[i] = T[i-1]
            i--
        }

        T[i] = temp
    }
}
func median(T tabInt, n int) int {
    if n%2 == 1 {
        return T[n/2]
    }

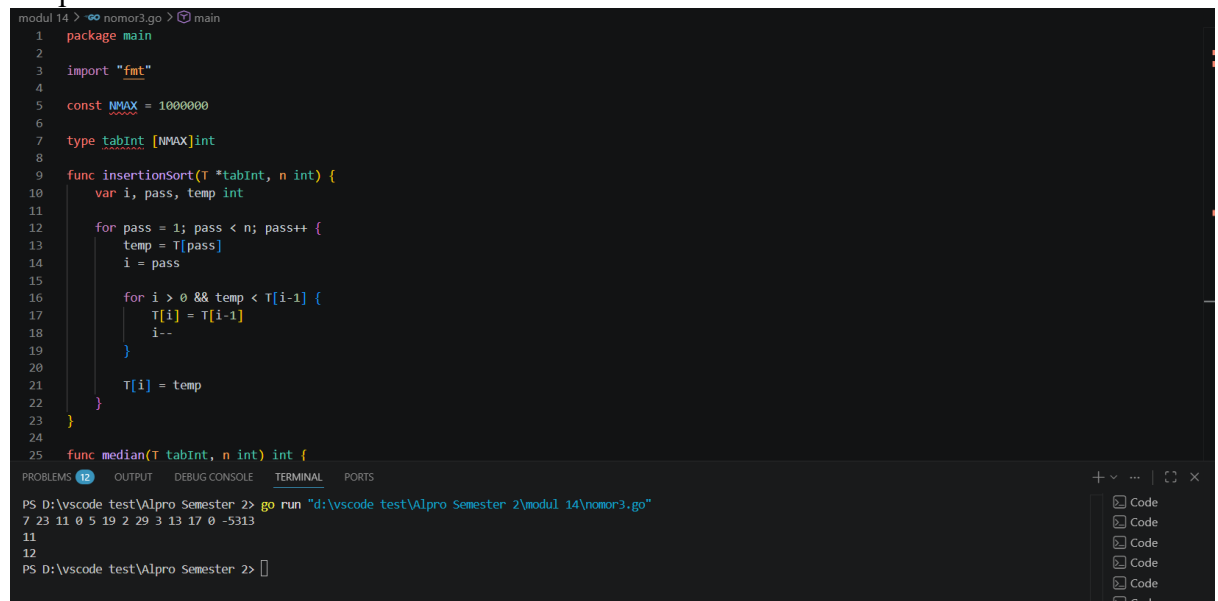
    return (T[(n/2)-1] + T[n/2]) / 2
}
func main() {
    var T tabInt
    var x, n int

    for {
        fmt.Scan(&x)

        if x == -5313 {
            break
        }

        if x == 0 {
            insertionSort(&T, n)
            fmt.Println(median(T, n))
        } else {
            T[n] = x
            n++
        }
    }
}
```

Output:



```
modul 14 > nomor3.go main
1 package main
2
3 import "fmt"
4
5 const NMAX = 1000000
6
7 type tabInt [NMAX]int
8
9 func insertionSort(T *tabInt, n int) {
10     var i, pass, temp int
11
12     for pass = 1; pass < n; pass++ {
13         temp = T[pass]
14         i = pass
15
16         for i > 0 && temp < T[i-1] {
17             T[i] = T[i-1]
18             i--
19         }
20
21         T[i] = temp
22     }
23 }
24
25 func median(T tabInt, n int) int {
26     insertionSort(&T, n)
27     if n%2 == 1 {
28         return T[n/2]
29     } else {
30         return (T[n/2-1] + T[n/2]) / 2
31     }
32 }
33
34 func main() {
35     var T tabInt
36     var n int
37     for {
38         fmt.Scanln(&n)
39         if n == 0 {
40             break
41         }
42         T[n-1] = 0
43     }
44     fmt.Println("Median:", median(T, n))
45 }
```

PS D:\vscode test\Alpro Semester 2> go run "d:\vscode test\Alpro Semester 2\modul 14\nomor3.go"

7 23 11 0 5 19 2 29 3 13 17 0 -5313

11

12

PS D:\vscode test\Alpro Semester 2>

Deskripsi Program:

Program ini dibuat untuk mencari dan menampilkan nilai median dari sekumpulan data yang sudah dibaca setiap kali input bernilai 0. Program membaca data bilangan bulat satu per satu hingga menemukan tanda akhir -5313. Setiap data selain 0 dan -5313 akan disimpan ke dalam array. Ketika program menemukan angka 0, seluruh data yang sudah tersimpan akan diurutkan menggunakan metode insertion sort dari kecil ke besar. Setelah data terurut, program menghitung median. Jika jumlah data ganjil, median diambil dari nilai tengah array, sedangkan jika jumlah data genap, median dihitung dari rata-rata dua nilai tengah lalu dibulatkan ke bawah karena menggunakan tipe integer. Setelah median ditampilkan, program melanjutkan pembacaan data berikutnya hingga menemukan marker akhir -5313.

Insertion Sort

1. Pengurutan Data dan Pengecekan Jarak Selisih Tetap (Insertion Sort)

Source Code:

```
package main
import "fmt"
const NMAX = 1000
type tabInt [NMAX]int
func insertionSort(T *tabInt, n int) {
    var i, pass, temp int

    for pass = 1; pass < n; pass++ {
        temp = T[pass]
        i = pass

        for i > 0 && T[i-1] > temp {
            T[i] = T[i-1]
            i--
        }

        T[i] = temp
    }
}
func cekJarak(T tabInt, n int) {
    var selisih, i int

    if n < 2 {
        fmt.Println("Data berjarak tidak tetap")
        return
    }

    selisih = T[1] - T[0]

    for i = 1; i < n; i++ {
        if T[i]-T[i-1] != selisih {
            fmt.Println("Data berjarak tidak tetap")
            return
        }
    }

    fmt.Println("Data berjarak", selisih)
}
func main() {
    var T tabInt
    var x, n int

    for {
        fmt.Scan(&x)
```

```

        if x < 0 {
            break
        }

        T[n] = x
        n++
    }
    insertionSort(&T, n)

    for i := 0; i < n; i++ {
        fmt.Print(T[i])
        if i < n-1 {
            fmt.Print(" ")
        }
    }
    fmt.Println()

    cekJarak(T, n)
}

```

Output:

The screenshot shows a Go program in VS Code. The source code defines an array T of size NMAX (1000) and implements an insertion sort algorithm. The main function reads input numbers until a negative number is encountered, then prints the sorted array. The terminal output shows the input sequence: 31 13 25 43 1 7 19 37 -5, followed by the sorted sequence: 1 7 13 19 25 31 37 43. It then shows the input sequence: 4 40 14 8 26 1 38 2 32 -31, followed by the sorted sequence: 1 2 4 8 14 26 32 38 40. The program correctly identifies the end of input when a negative number is reached.

```

modul 14 > go run "d:\vscode test\Alpro Semester 2\modul 14\nomor4.go"
1 package main
2
3 import "fmt"
4
5 const NMAX = 1000
6
7 type tabInt [NMAX]int
8
9 func insertionSort(T *tabInt, n int) {
10     var i, pass, temp int
11
12     for pass = 1; pass < n; pass++ {
13         temp = T[pass]
14         i = pass
15
16         for i > 0 && T[i-1] > temp {
17             T[i] = T[i-1]
18             i--
19         }
20
21         T[i] = temp
22     }
23 }
24
25 func cekJarak(T tabInt, n int) {
26     var selisih, i int

```

```

PS D:\vscode test\Alpro Semester 2> go run "d:\vscode test\Alpro Semester 2\modul 14\nomor4.go"
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6
PS D:\vscode test\Alpro Semester 2> go run "d:\vscode test\Alpro Semester 2\modul 14\nomor4.go"
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap
PS D:\vscode test\Alpro Semester 2>

```

Deskripsi Program:

Program ini digunakan untuk membaca sekumpulan data bilangan bulat yang diinput oleh pengguna, kemudian hanya menyimpan bilangan non-negatif ke dalam array hingga ditemukan bilangan negatif sebagai tanda akhir input. Setelah data terkumpul, program mengurutkan seluruh data menggunakan metode insertion sort dari kecil ke besar. Setelah itu, program menampilkan hasil data yang sudah terurut.

Selanjutnya, program memeriksa apakah selisih antar elemen bernilai tetap (membentuk pola deret aritmatika). Jika semua selisih sama, maka program menampilkan “Data berjarak x”, dengan x adalah selisih antar elemen. Jika tidak sama, maka program menampilkan “Data berjarak tidak tetap”.

insertionSort(&T, n)

Mengurutkan seluruh data dari kecil ke besar menggunakan insertion sort.

selisih = T[1] - T[0]

Menentukan selisih awal antar data untuk pengecekan pola.

```
if T[i]-T[i-1] != selisih {  
    fmt.Println("Data berjarak tidak tetap")  
    return  
}
```

Mengecek apakah semua selisih sama, jika berbeda langsung dianggap tidak tetap

2. Perpustakaan digunakan untuk mengelola data buku di dalam suatu perpustakaan

Source Code:

```
package main

import "fmt"

const nMax = 7919

type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating      int
}

type DaftarBuku [nMax]Buku

func DaftarkanBuku(p *DaftarBuku, n int) {
    for i := 0; i < n; i++ {
        fmt.Scan(&p[i].id, &p[i].judul, &p[i].penulis,
            &p[i].penerbit,
            &p[i].eksemplar, &p[i].tahun, &p[i].rating)
    }
}

func CetakTerfavorit(p DaftarBuku, n int) {
    idx := 0

    for i := 1; i < n; i++ {
        if p[i].rating > p[idx].rating {
            idx = i
        }
    }

    fmt.Println(p[idx].judul, p[idx].penulis,
        p[idx].penerbit, p[idx].tahun)
}

func UrutBuku(p *DaftarBuku, n int) {
    for i := 1; i < n; i++ {
        temp := p[i]
        j := i

        for j > 0 && p[j-1].rating < temp.rating {
            p[j] = p[j-1]
            j--
        }

        p[j] = temp
    }
}
```

```

func Cetak5Terbaru(p DaftarBuku, n int) {
    limit := 5
    if n < 5 {
        limit = n
    }

    for i := 0; i < limit; i++ {
        fmt.Println(p[i].judul)
    }
}

func CariBuku(p DaftarBuku, n, r int) {
    low, high := 0, n-1

    for low <= high {
        mid := (low + high) / 2

        if p[mid].rating == r {
            fmt.Println(p[mid].judul, p[mid].penulis,
p[mid].penerbit,
            p[mid].tahun, p[mid].eksemplar,
p[mid].rating)
            return
        } else if p[mid].rating < r {
            high = mid - 1
        } else {
            low = mid + 1
        }
    }

    fmt.Println("Tidak ada buku dengan rating seperti
itu")
}

func main() {
    var p DaftarBuku
    var n, r int

    fmt.Scan(&n)

    DaftarkanBuku(&p, n)

    CetakTerfavorit(p, n)

    UrutBuku(&p, n)

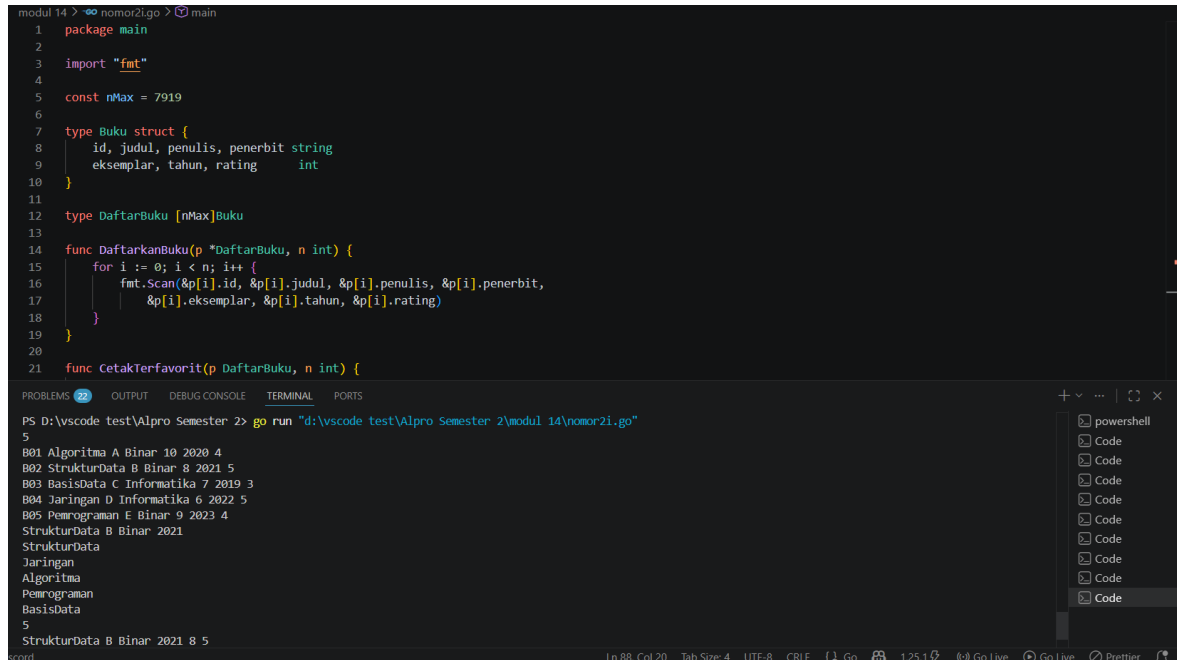
    Cetak5Terbaru(p, n)

    fmt.Scan(&r)
    CariBuku(p, n, r)
}

```

```
}
```

Output:



```
modul 14 > go run nomor21.go
1 package main
2
3 import "fmt"
4
5 const nMax = 7919
6
7 type Buku struct {
8     id, judul, penulis, penerbit string
9     eksemplar, tahun, rating      int
10 }
11
12 type DaftarBuku [nMax]Buku
13
14 func DaftarkanBuku(p *DaftarBuku, n int) {
15     for i := 0; i < n; i++ {
16         fmt.Scan(&p[i].id, &p[i].judul, &p[i].penulis, &p[i].penerbit,
17                 &p[i].eksemplar, &p[i].tahun, &p[i].rating)
18     }
19 }
20
21 func CetakTerfavorit(p DaftarBuku, n int) {
```

```
PS D:\vscode test\Alpro Semester 2> go run "d:\vscode test\Alpro Semester 2\modul 14\nomor21.go"
5
B01 Algoritma A Binar 10 2020 4
B02 StrukturData B Binar 8 2021 5
B03 BasisData C Informatika 7 2019 3
B04 Jaringan D Informatika 6 2022 5
B05 Pemrograman E Binar 9 2023 4
StrukturData B Binar 2021
StrukturData
Jaringan
Algoritma
Pemrograman
BasisData
5
StrukturData B Binar 2021 8 5
```

Deskripsi Program:

Program ini digunakan untuk mengelola data buku dalam sebuah perpustakaan. Program menerima masukan berupa jumlah data buku (N), kemudian N baris data buku yang berisi informasi lengkap seperti id, judul, penulis, penerbit, jumlah eksemplar, tahun terbit, dan rating buku. Setelah seluruh data dimasukkan, program akan melakukan beberapa proses pengolahan data.

Pertama, program mencari buku terfavorit yaitu buku dengan rating tertinggi, kemudian menampilkan informasi buku tersebut (judul, penulis, penerbit, dan tahun). Selanjutnya, data buku diurutkan berdasarkan rating secara menurun (dari tertinggi ke terendah) menggunakan metode insertion sort. Setelah itu, program menampilkan lima judul buku dengan rating tertinggi. Terakhir, program melakukan pencarian buku berdasarkan rating yang dimasukkan pada baris terakhir menggunakan metode pencarian (binary search), dan menampilkan seluruh data buku tersebut jika ditemukan. Jika tidak ada buku dengan rating yang sesuai, maka program akan menampilkan pesan bahwa buku tidak ditemukan.

DAFTAR PUSTAKA

<https://www.youtube.com/playlist?list=PLjRBWix725xrVtr7q2Asm5LdTmTNSbH8A>